Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Alexander Antonio Sánchez Azuero
Asignatura	Sistemas Distribuidos
Ciclo	6 A
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	005
Título de la Práctica	Mensajería Distribuida con RabbitMQ y MQTT
Nombre del Docente	Ing. José Oswaldo Guamán Quinche
Fecha	Martes 20 y viernes 23 de octubre de 2026
Horario	Martes: 10h30 – 11h30 Viernes: 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Implementar la capa IoT de un sistema de monitoreo de flota logística utilizando el protocolo MQTT: configurar un broker Mosquitto en Docker, simular sensores que publican telemetría en topics jerárquicos con paho-mqtt, implementar un suscriptor que almacene los datos en SQLite y genere alertas por umbral, y construir un servicio puente que publique las alertas en colas de RabbitMQ.

Objetivos específicos:

- Comprender los fundamentos del protocolo MQTT y su aplicación en entornos IoT.
- Configurar e instalar un broker Mosquitto para mensajería publicador/suscriptor.
- Implementar publicadores y suscriptores MQTT en Python usando paho-mqtt.
- Diseñar jerarquías de topics para organizar la telemetría de una flota vehicular.
- Entender el protocolo AMQP 0-9-1 y el modelo de intercambio de RabbitMQ.
- Configurar exchanges, colas y bindings en RabbitMQ.

- Integrar MQTT y RabbitMQ mediante un servicio puente en Python.

3. Materiales, Reactivos, Equipos y Herramientas

Se usaron los recursos indicados en la guía para la Parte 1. Java JDK 17, Spring Boot y Postman no se utilizaron todavía porque corresponden a la Parte 2 (microservicios). También se agregaron algunas herramientas de apoyo que se detallan a continuación.

Software:

- Guía de Actividades Práctico-Experimentales Nro. 005 entregada por el docente.
- Python 3.14 con un entorno virtual (.venv) propio del proyecto.
- paho-mqtt 2.1.0, cliente MQTT para Python.
- pika 1.4.4, cliente AMQP para Python.
- Docker y Docker Compose.
- Eclipse Mosquitto 2.0 (imagen eclipse-mosquitto:2.0).
- RabbitMQ 3.13.7 con el plugin de administración (imagen rabbitmq:3-management).
- SQLite 3: módulo sqlite3 de Python y consola sqlite3 (versión 3.54).
- Terminal zsh y editor de código para escribir los scripts.
- Navegador web para ingresar al panel de RabbitMQ en el puerto 15672.

Hardware:

- Computador personal con sistema operativo CachyOS (Linux).

4. Procedimiento / Metodología Ejecutada

Todo el trabajo lo organicé en una carpeta llamada mqtt-flota dentro de PracticasDistribuidos y lo fui armando por partes, probando cada componente antes de pasar al siguiente. Los pasos que seguí fueron estos:

Preparación del broker Mosquitto

Primero escribí el archivo docker-compose.yml con el servicio de Mosquitto (imagen eclipse-mosquitto:2.0, contenedor mosquitto_broker y puerto 1883 publicado) y el archivo mosquitto/config/mosquitto.conf, donde habilité las conexiones anónimas y la persistencia de datos. A diferencia de la guía, agregué un tercer volumen para guardar el log del broker en mosquitto/log, porque así podía revisar las conexiones cuando algo no funcionaba. Levanté el contenedor con docker compose up -d y comprobé que estuviera activo con docker ps y con el comando ss, que mostró el puerto 1883 en escucha. Para la prueba rápida abrí dos terminales: en una dejé mosquitto_sub escuchando test/topic y desde la otra envié el texto "Hola MQTT", que llegó sin problema.

Simulador de sensores

Creé un entorno virtual en la carpeta del proyecto e instalé paho-mqtt y pika. Luego programé simulador_sensores.py, que genera datos para tres vehículos (VH-001, VH-002 y VH-003). Cada vehículo envía su posición GPS con la velocidad, la temperatura del compartimento de carga y el nivel de combustible, en formato JSON junto con el identificador del vehículo y la hora en UTC. Los mensajes se publican en topics del tipo flota/VH-001/gps con QoS 1, y el programa espera 5 segundos entre un vehículo y otro.

Como el ciclo infinito de la guía hacía difícil obtener evidencias ordenadas, le añadí el parámetro --rondas para indicar cuántas vueltas debe dar el simulador; con una ronda se publican 9 mensajes. También controlé la interrupción con Ctrl + C y los errores de conexión, cerrando el cliente al terminar. Antes de ejecutarlo revisé la sintaxis con python -m py_compile y después confirmé con mosquitto_sub sobre flota/# que los nueve mensajes llegaban al broker.

Suscriptor con almacenamiento en SQLite

El script suscriptor_sqlite.py crea la base telemetria.db con las tablas gps_data, temp_data y fuel_data, y se suscribe a flota/+/gps, flota/+/temperatura y flota/+/combustible. Según el topic de cada mensaje guarda los datos en la tabla que corresponde y revisa los umbrales: si la temperatura pasa de 4 °C muestra una alerta de temperatura, y si el combustible baja del 20 % muestra una alerta de combustible.

Con el suscriptor corriendo ejecuté el simulador con dos rondas, lo que dejó 6 registros en cada tabla. Para revisar lo guardado escribí un script corto, verificar_resultados.py, que cuenta los registros y lista los últimos valores, y además lo comprobé directamente desde la consola de sqlite3.

RabbitMQ y servicio puente

En el mismo docker-compose.yml agregué el servicio rabbitmq con la imagen rabbitmq:3-management, los puertos 5672 (AMQP) y 15672 (panel web) y un volumen con nombre para sus datos. Revisé los logs del contenedor, hice un ping con rabbitmq-diagnostics y entré al panel web con el usuario guest.

Después escribí mqtt_rabbitmq_bridge.py. Al arrancar se conecta a RabbitMQ con pika, declara como durables las colas cola.gps.telemetria, cola.alertas.temperatura y cola.combustible.nivel, y se suscribe a flota/# en Mosquitto. Los datos GPS pasan directo a su cola, mientras que la temperatura y el combustible solo se reenvían cuando superan el umbral, agregando el tipo de alerta (TEMP_ALERT o FUEL_LOW) y un mensaje descriptivo. Lo ejecuté con python -u para ver la salida en el momento y confirmé en el panel que las tres colas aparecían con la marca D de durable.

Pruebas controladas

Como los valores aleatorios no siempre generan alertas, preparé prueba_alertas.py, que publica para VH-001 una temperatura de 7 °C y un combustible de 15 %, valores elegidos para que se activen las dos reglas. Al final guardé las dependencias del entorno en requirements.txt con pip freeze. Después de las correcciones que se describen a continuación, repetí esta prueba el 2 de noviembre con el suscriptor y el puente en ejecución.

Correcciones y ajustes durante la implementación

Al repetir las pruebas aparecieron algunos problemas que no se habían notado en las primeras ejecuciones. Los fui resolviendo en este orden:

Carpeta de trabajo incompleta. La carpeta PracticasDistribuidos/mqtt-flota solo conservaba la carpeta mosquitto y le faltaban los scripts de Python, así que no se podía ejecutar el simulador, el suscriptor, el puente ni las pruebas. Pasé a trabajar en la copia completa del proyecto, PracticasDistribuidos-Terminada/mqtt-flota; por eso las capturas finales muestran esa ruta.

Configuración de Mosquitto. El contenedor mostraba el error "Unable to open config file /mosquitto/config/mosquitto.conf" porque seguía montado sobre la

carpeta incompleta. En la copia completa el archivo mosquitto.conf tiene este contenido:

```
listener 1883 0.0.0.0
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log
```

La línea listener 1883 0.0.0.0 es importante en Mosquitto 2.0, porque sin ella el broker solo acepta conexiones que salen del mismo contenedor. Eliminé únicamente el contenedor de Mosquitto con docker rm -f mosquitto_broker y lo volví a crear desde la carpeta correcta con docker compose up -d mosquitto. RabbitMQ y sus datos no se tocaron.

Entorno virtual de Python. El entorno anterior no tenía bien instaladas las librerías, así que creé uno nuevo con python -m venv .venv, lo activé e instalé las dependencias con pip install -r requirements.txt. Después comprobé que estuvieran paho-mqtt 2.1.0 y pika 1.4.4.

Desconexión del puente con RabbitMQ. Fue el problema más importante. El puente terminaba con cliente.loop_forever(), que deja todo el control a paho-mqtt, mientras la conexión de pika (BlockingConnection) se quedaba sin atender. RabbitMQ espera unas señales periódicas llamadas heartbeats para saber que el cliente sigue activo; como no le llegaban, cerró la conexión y en sus logs apareció "missed heartbeats from client, timeout: 60s". En el puente esto se veía como los errores "Transport indicated EOF" y "Channel is closed". Para corregirlo reemplacé loop_forever() por un ciclo que atiende las dos conexiones por turnos:

```
while True:
    cliente.loop(timeout=1.0)
    conexion.process_data_events(time_limit=0)
```

La primera instrucción procesa los mensajes que llegan por MQTT y la segunda deja que pika responda los heartbeats y envíe lo que tenga pendiente. Con este cambio el puente se mantuvo conectado y publicó en RabbitMQ sin errores.

Con todo corregido repetí la prueba controlada. En la verificación de la base cada tabla quedó con 8 registros (24 en total) y en RabbitMQ quedaron 3 mensajes listos, uno por cola. Las capturas de esta prueba están en las figuras 25 a 31.

5. Resultados

A continuación se presentan las capturas obtenidas durante la práctica, ordenadas según las actividades de la guía.

Actividad 1: entorno de desarrollo y broker Mosquitto

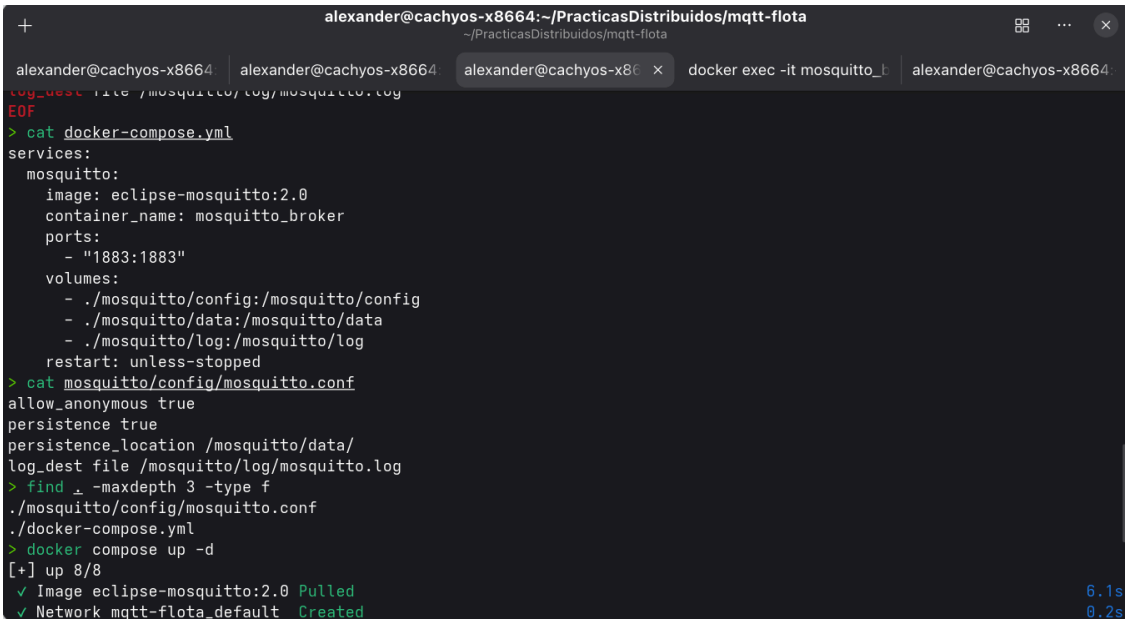


Fig. 1 *- Contenido de docker-compose.yml y mosquitto.conf, y arranque del contenedor con docker compose up -d.*

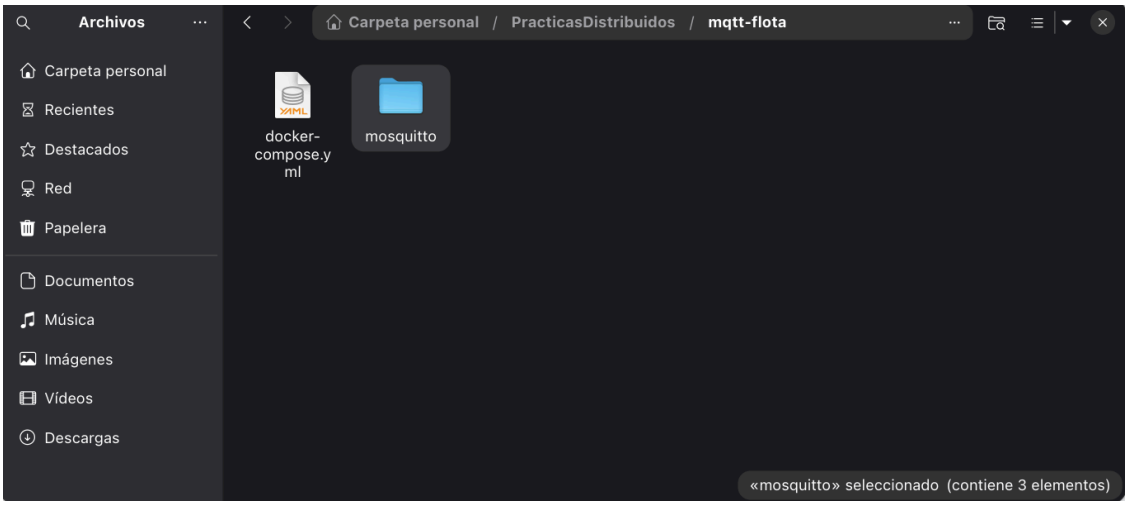


Fig. 2 *- Estructura inicial de la carpeta mqtt-flota.*

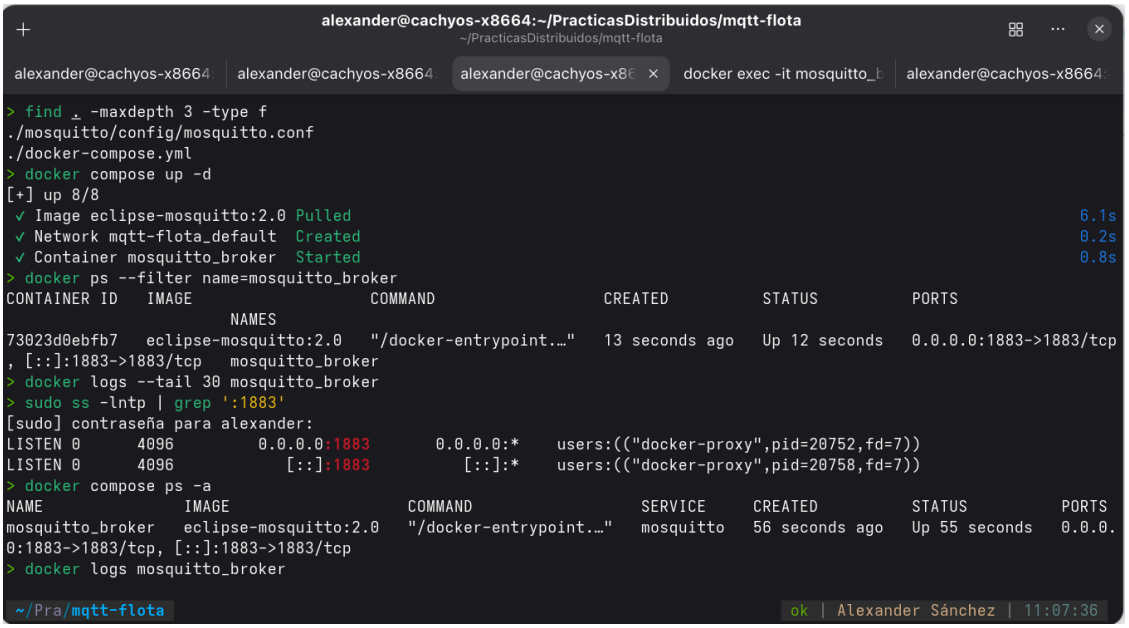


Fig. 3 *- Contenedor mosquitto_broker en ejecución y puerto 1883 en escucha.*

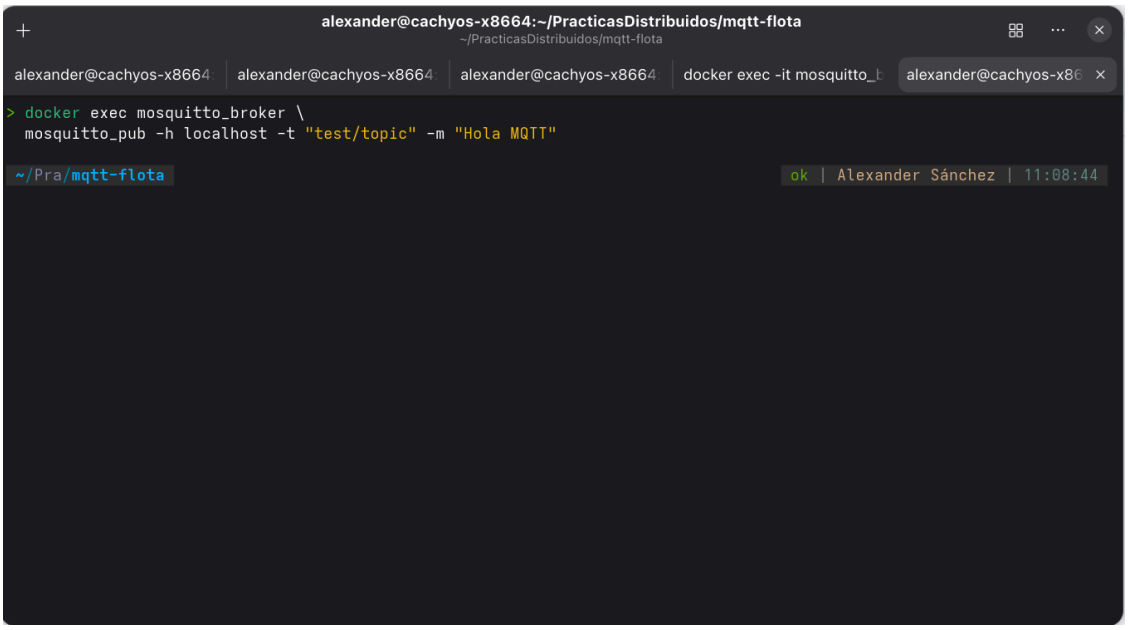


Fig. 4 *- Publicación del mensaje de prueba "Hola MQTT" en el topic test/topic.*

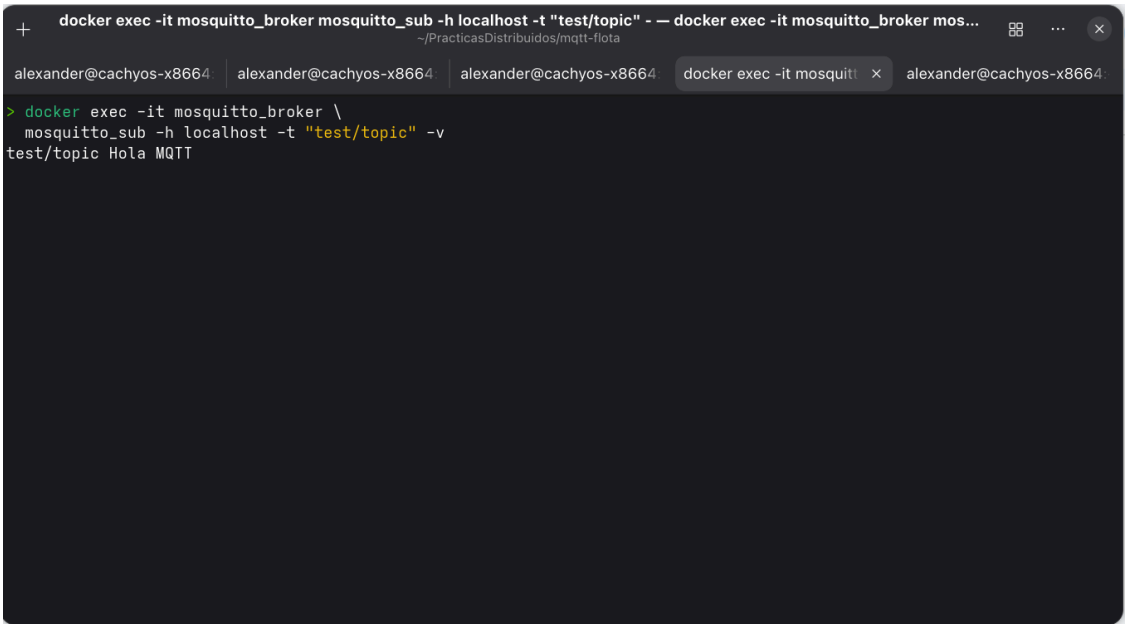


Fig. 5 *- El suscriptor recibe el mensaje "Hola MQTT".*

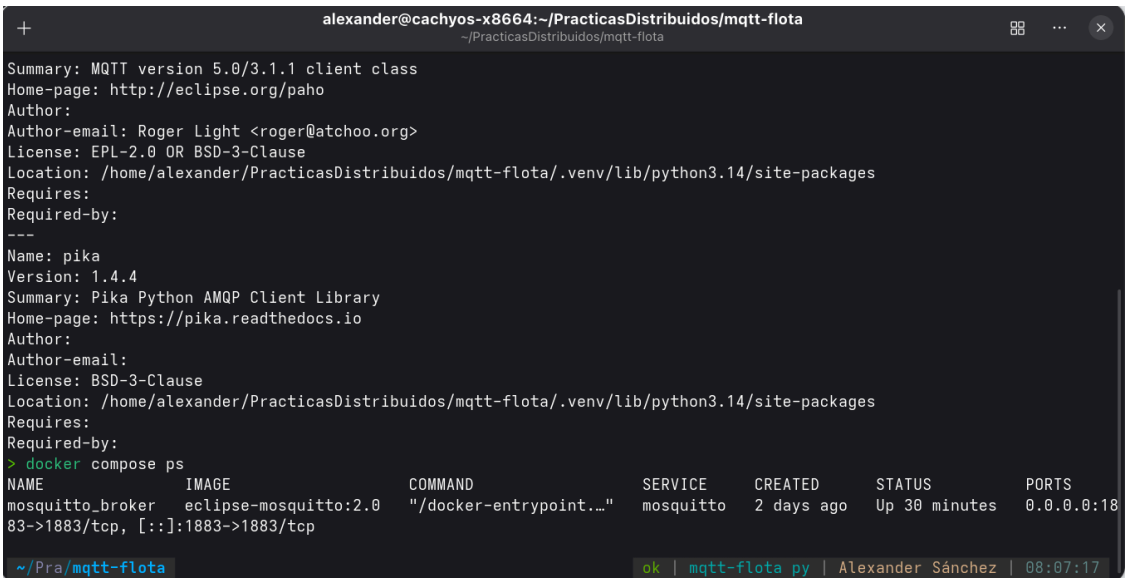


Fig. 6 *- Verificación del entorno: paho-mqtt y pika instalados y broker activo.*

El broker respondió desde el primer intento y el mensaje de prueba llegó al suscriptor, así que quedó listo para recibir la telemetría.

Actividad 2: simulador de sensores IoT

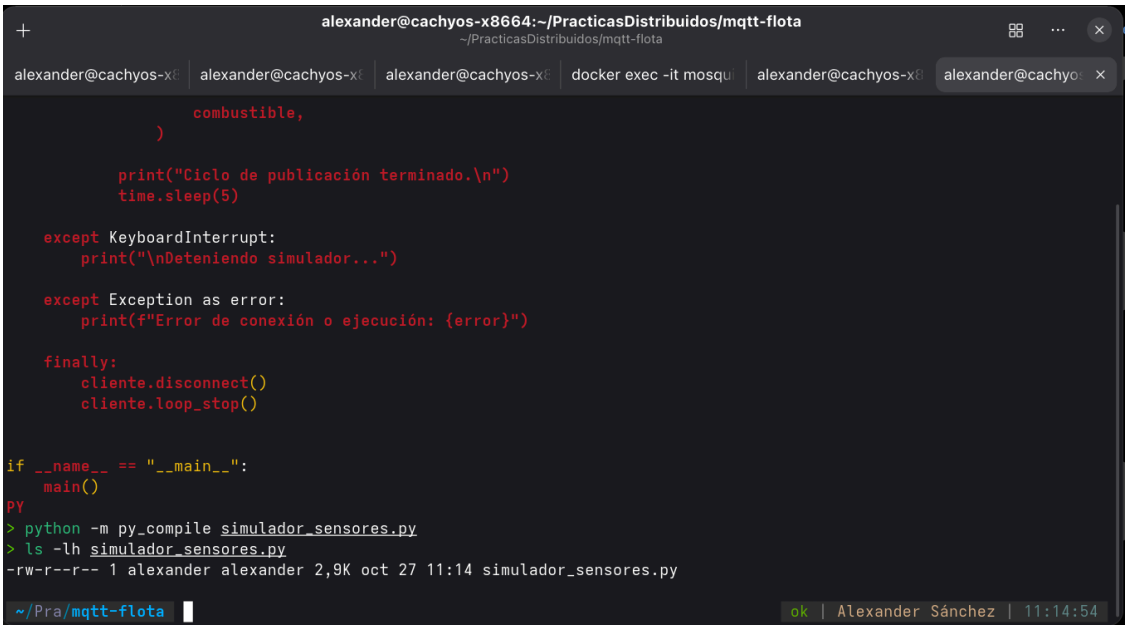


Fig. 7 *- Parte final de simulador_sensores.py y validación de sintaxis con py_compile.*

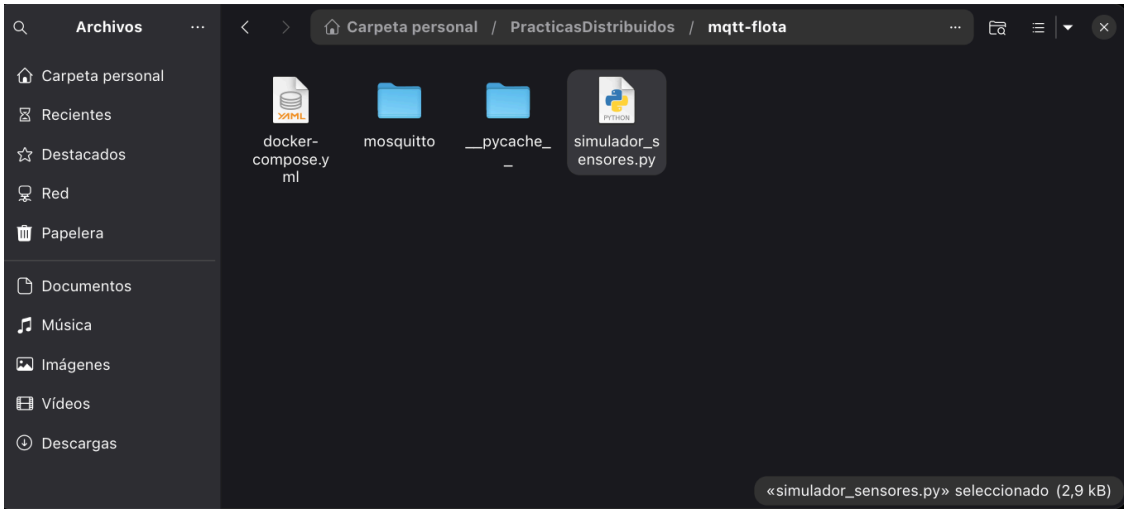


Fig. 8 *- Carpeta del proyecto con el archivo simulador_sensores.py.*

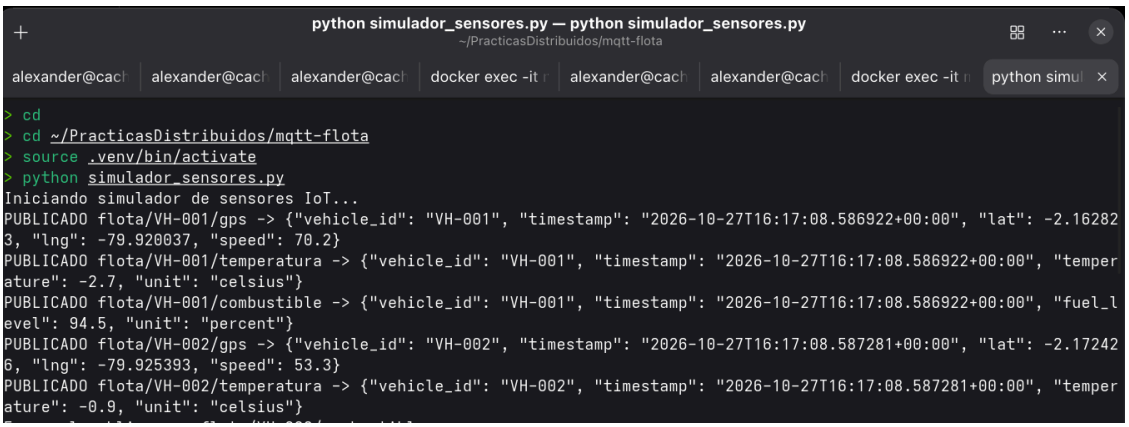


Fig. 9 *- Primera ejecución del simulador publicando en los topics flota/VH-00X.*

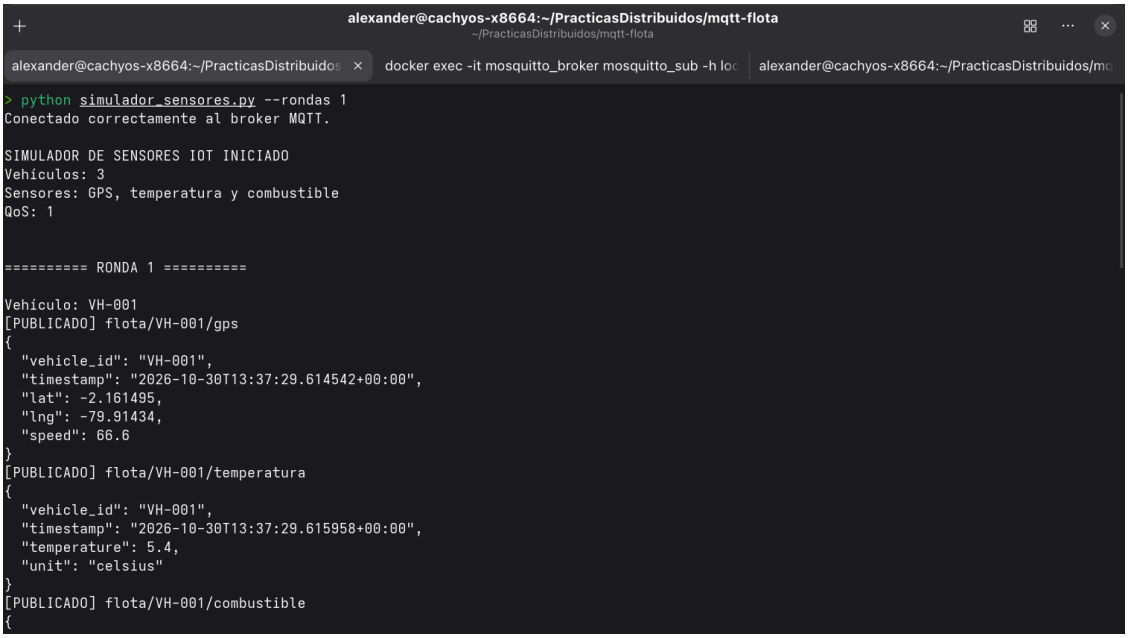


Fig. 10 *- Simulador con una ronda: conexión al broker y datos de VH-001.*

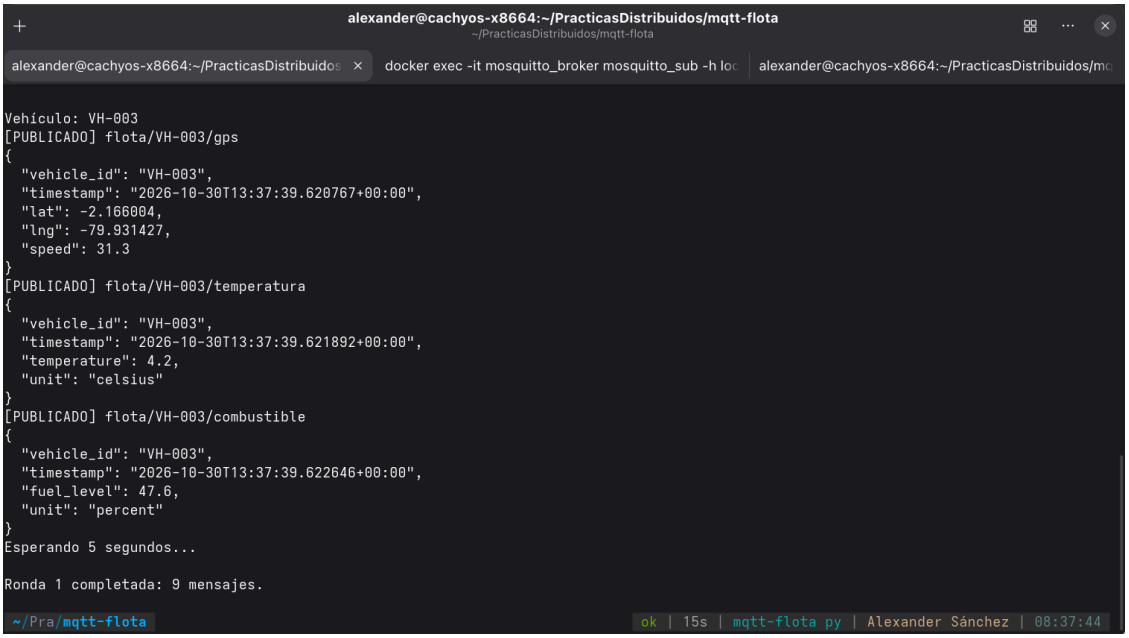


Fig. 11 *- Fin de la ronda con 9 mensajes publicados.*

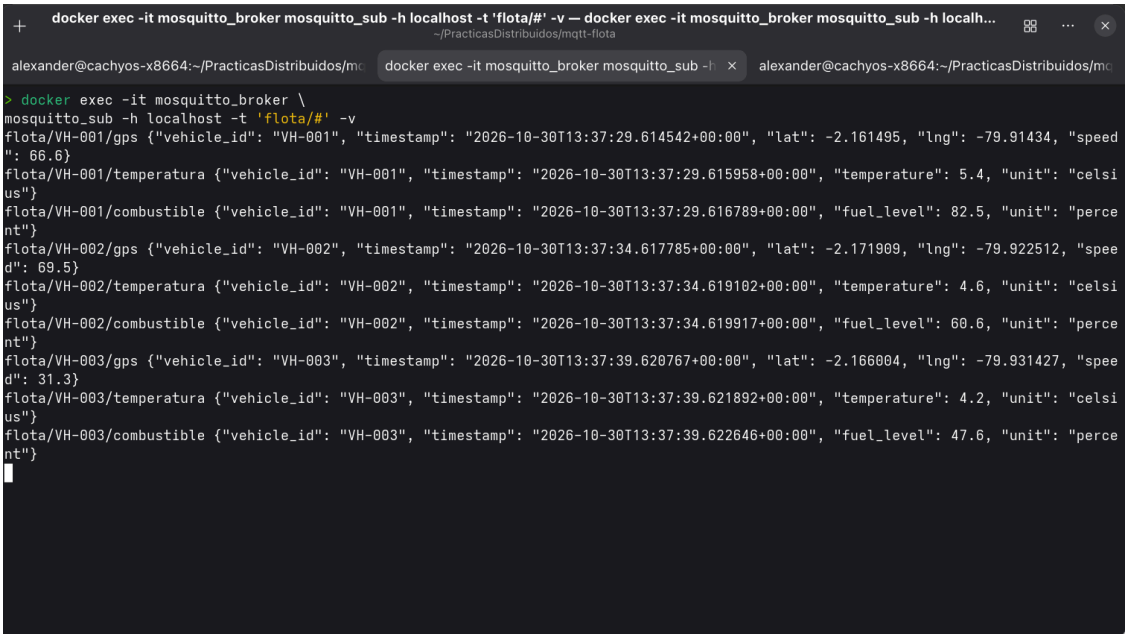


Fig. 12 *- Mensajes recibidos en el broker con mosquitto_sub sobre flota/#.*

Cada ronda produjo nueve mensajes (tres sensores por cada uno de los tres vehículos) y todos aparecieron en el broker con su topic y su JSON completo.

Actividad 3: suscriptor MQTT y almacenamiento en SQLite

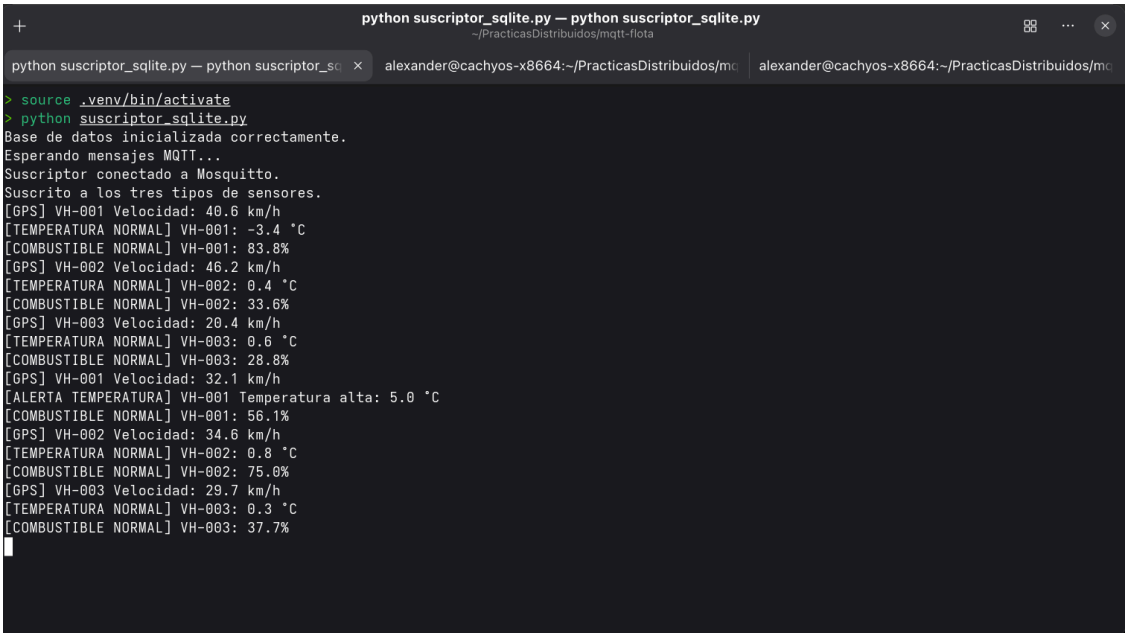


Fig. 13 *- Suscriptor recibiendo la telemetría y mostrando alerta de temperatura alta en VH-001 (5.0 °C).*

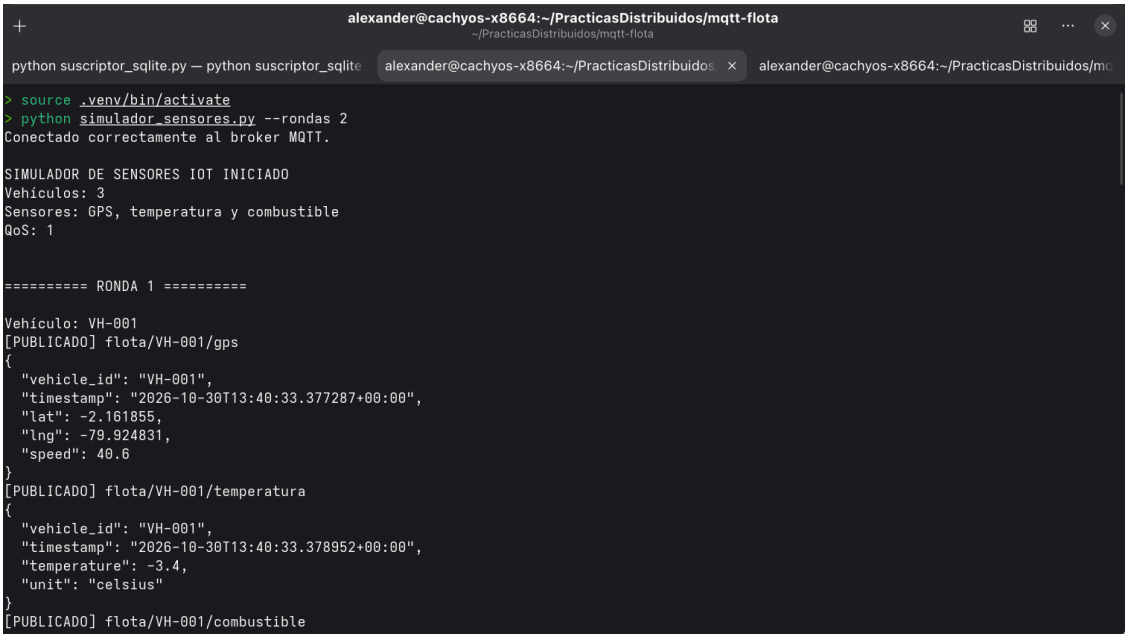


Fig. 14 *- Simulador ejecutado con dos rondas mientras el suscriptor estaba activo.*

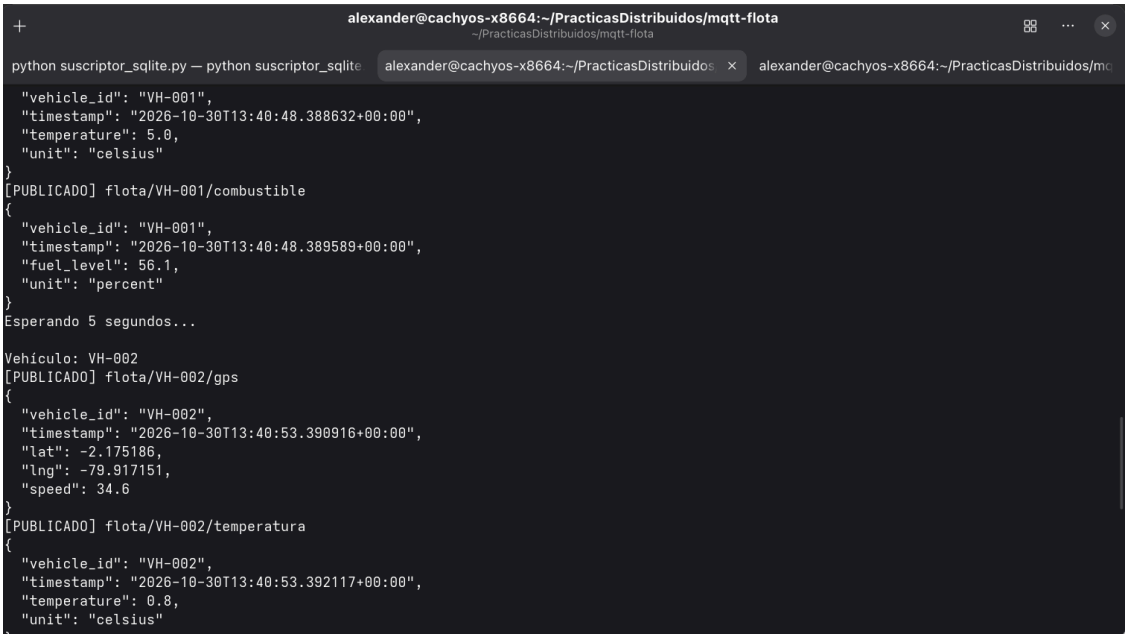


Fig. 15 *- Lectura de 5.0 °C en VH-001, que supera el umbral de 4 °C.*



Fig. 16 *- Segunda ronda completada con 9 mensajes.*

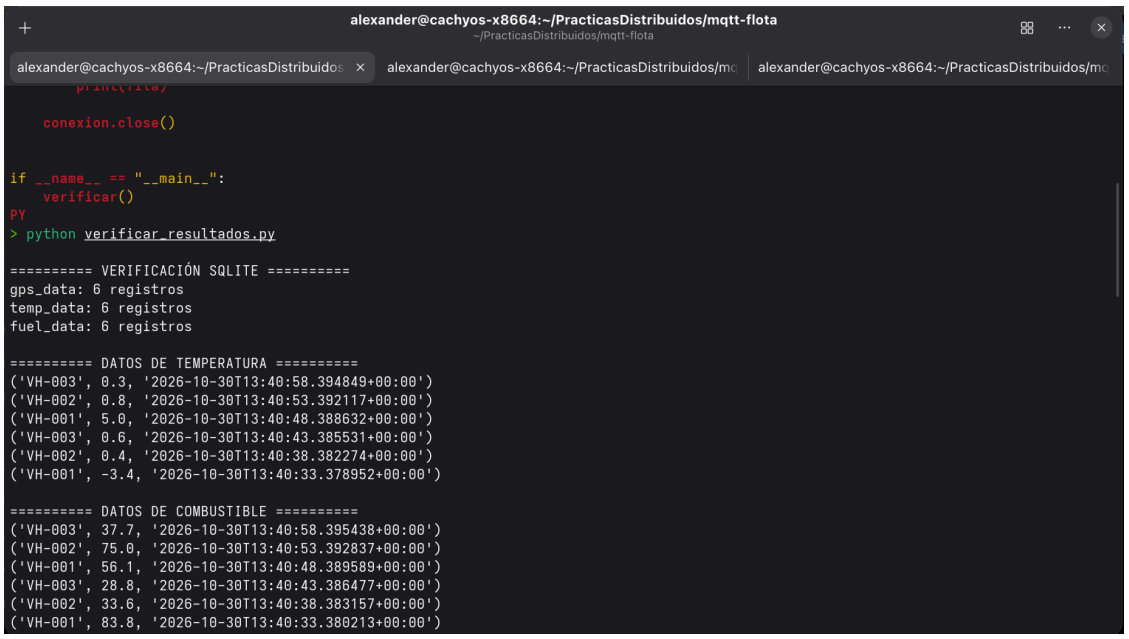


Fig. 17 *- Resultado de verificar_resultados.py con 6 registros por tabla.*

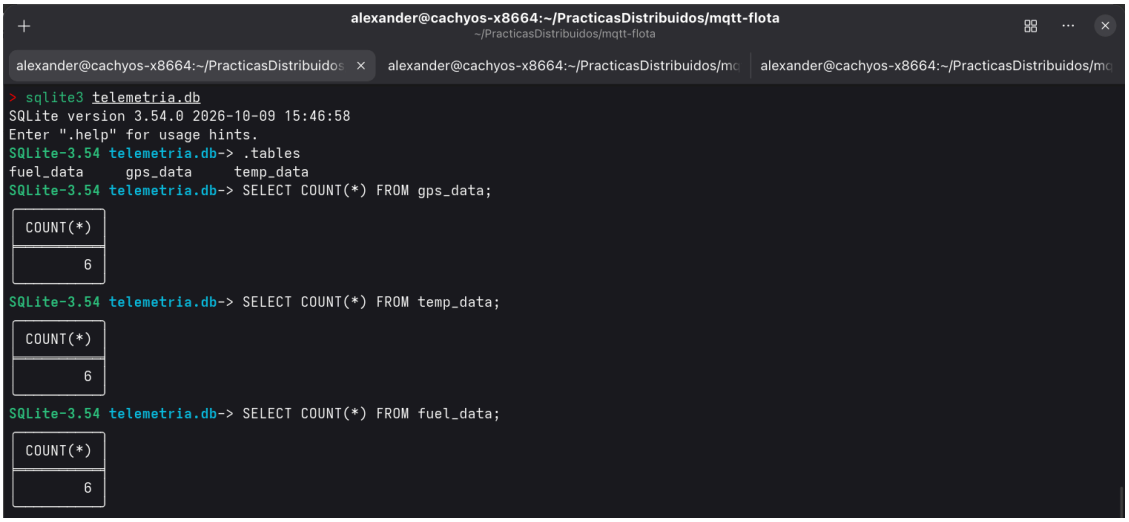


Fig. 18 *- Tablas de telemetria.db y conteo de registros desde la consola sqlite3.*

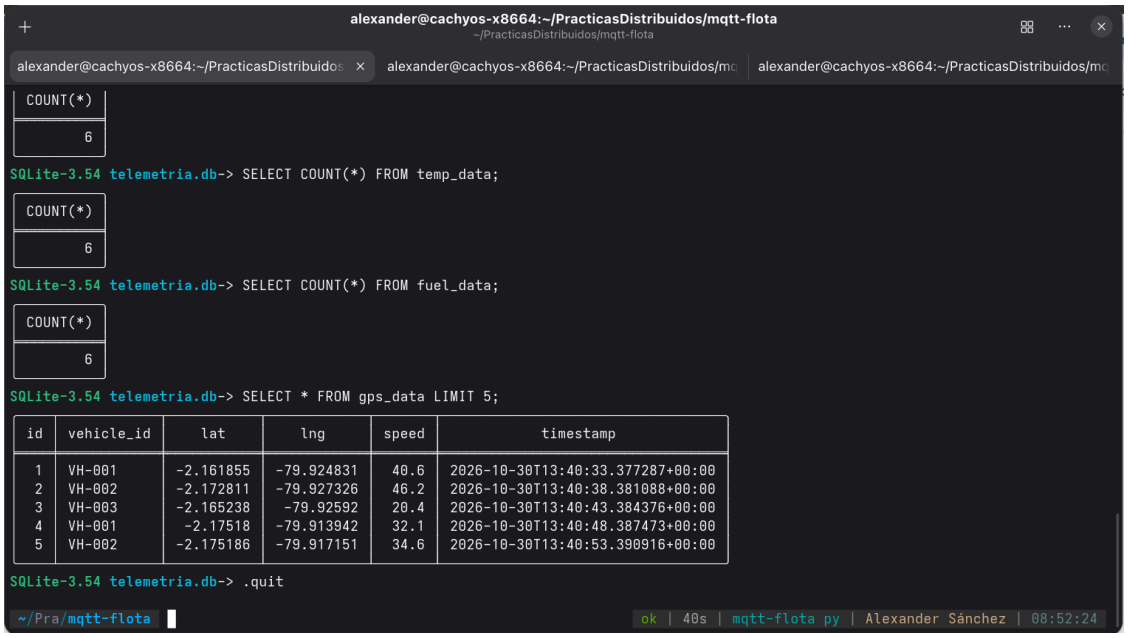


Fig. 19 *- Consulta de los primeros registros de la tabla gps_data.*

En las dos rondas se guardaron 18 lecturas, seis por tabla, y la única que superó un umbral fue la temperatura de VH-001 en la segunda ronda. Ningún vehículo bajó del 20 % de combustible en esa prueba, por eso la alerta de combustible se forzó después con valores controlados.

Actividad 4: RabbitMQ y servicio puente MQTT-RabbitMQ

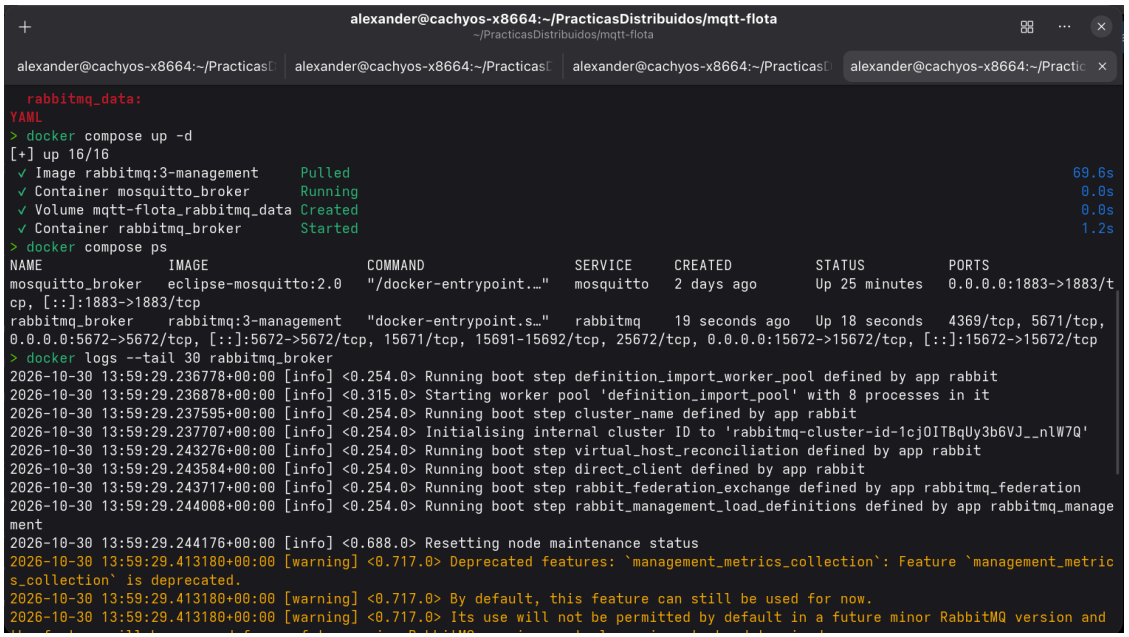


Fig. 20 *- Servicio RabbitMQ agregado a Docker Compose y contenedores en ejecución.*

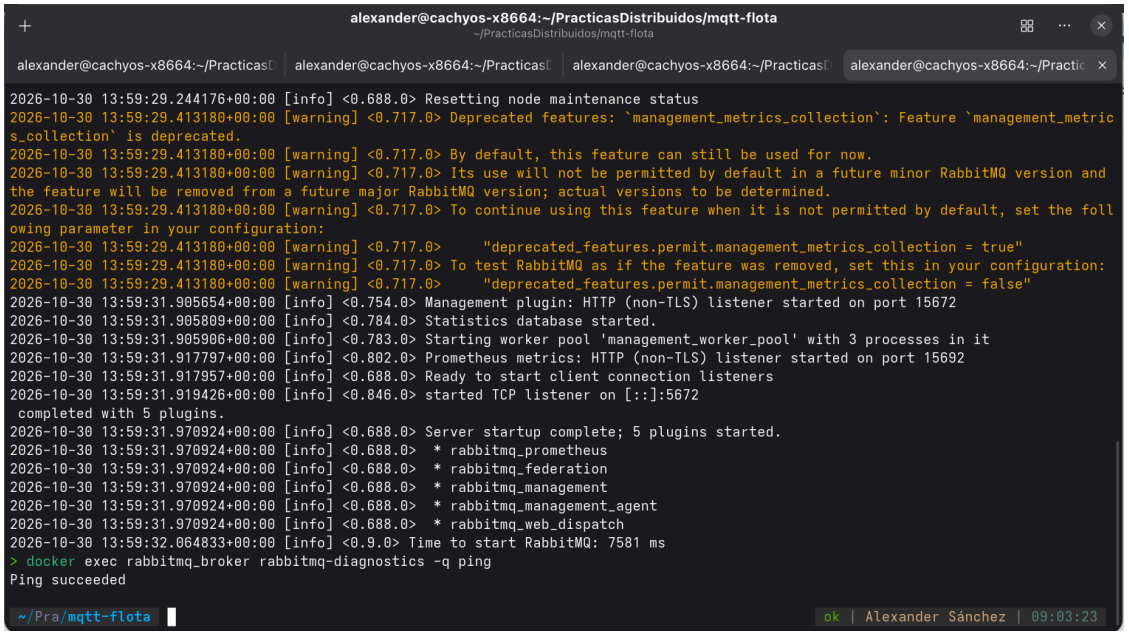


Fig. 21 *- Logs de arranque de RabbitMQ y prueba con rabbitmq-diagnostics ping.*

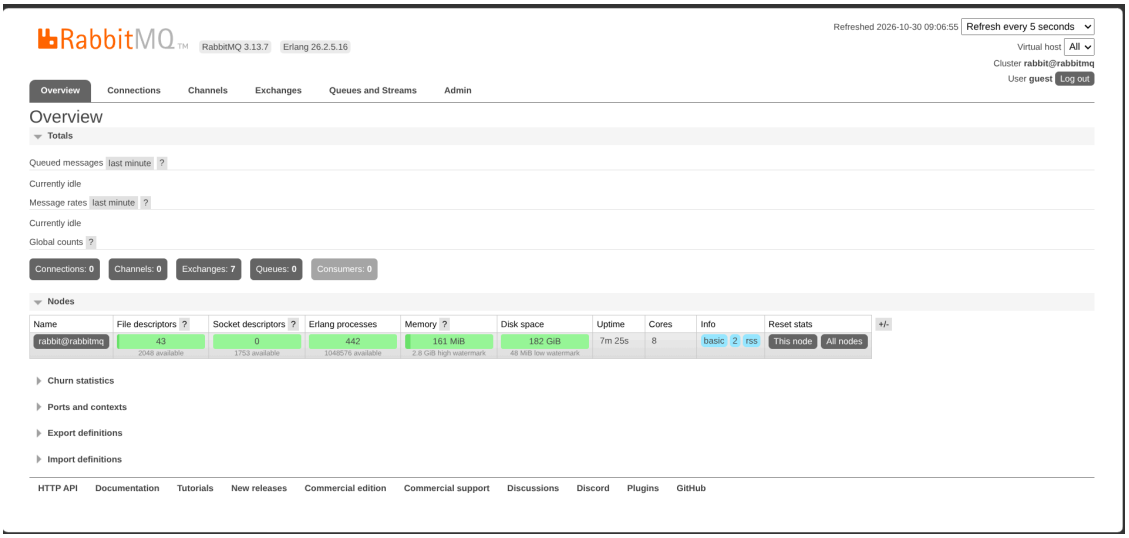


Fig. 22 *- Panel web de administración de RabbitMQ en el puerto 15672.*

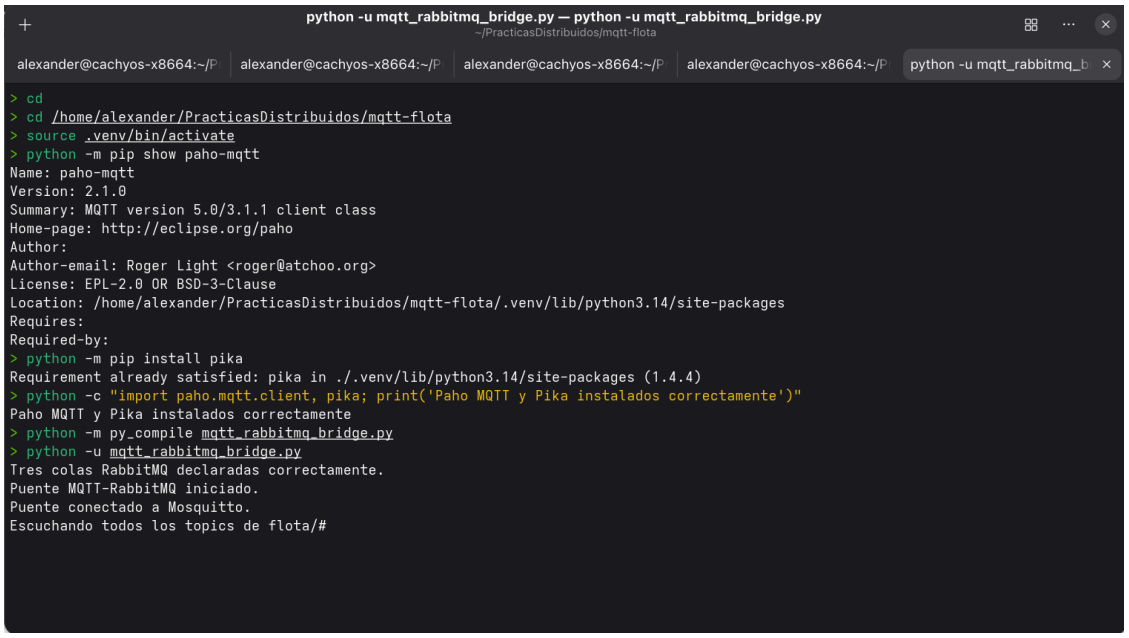


Fig. 23 *- Puente mqtt_rabbitmq_bridge.py en ejecución, escuchando flota/#.*

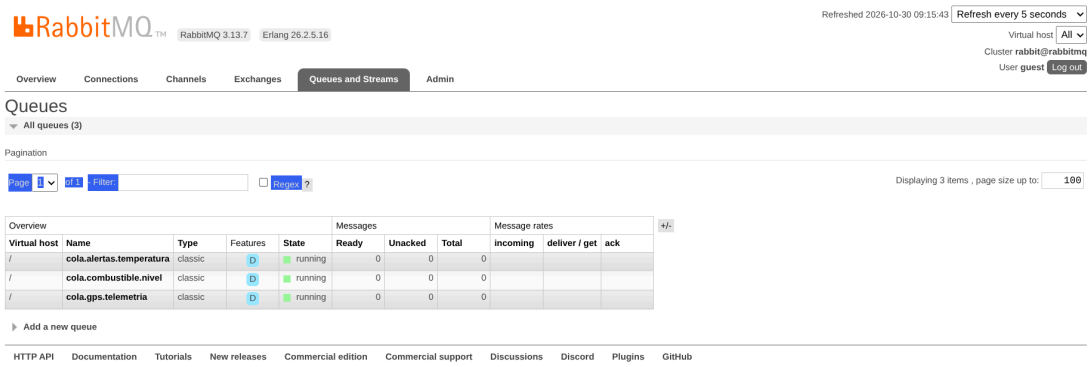


Fig. 24 *- Colas durables creadas por el puente en RabbitMQ.*

Pruebas finales después de las correcciones

Después de corregir los problemas descritos en el procedimiento, el 2 de noviembre volví a ejecutar todo desde la carpeta PracticasDistribuidos-Terminada/mqtt-flota, dejando el suscriptor y el puente activos antes de lanzar la prueba controlada.

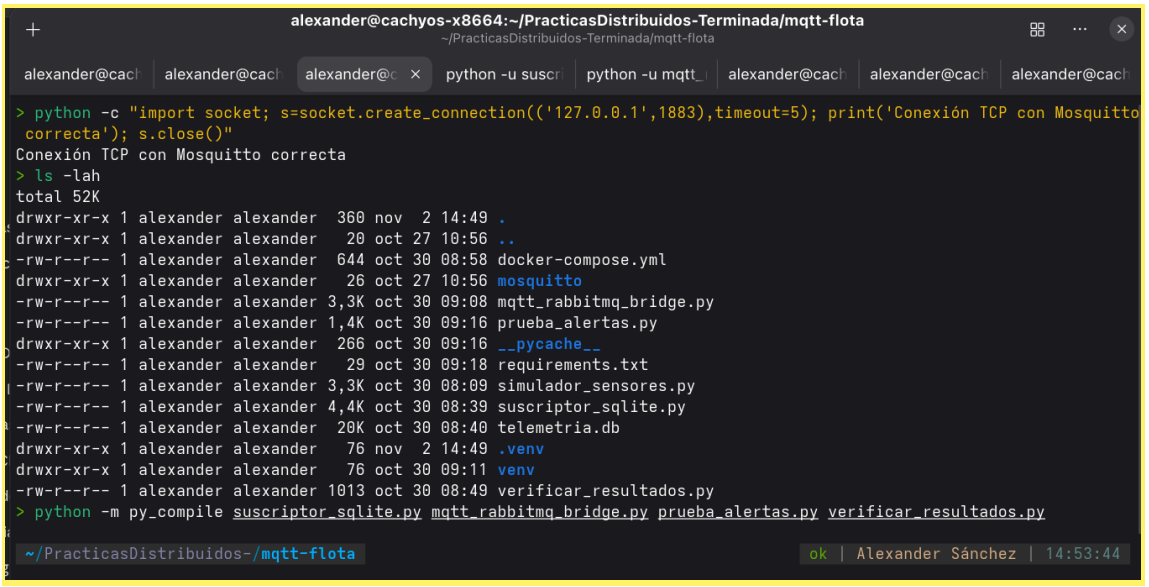


Fig. 25 *- Conexión TCP con Mosquitto en el puerto 1883 y archivos del proyecto en la carpeta corregida.*

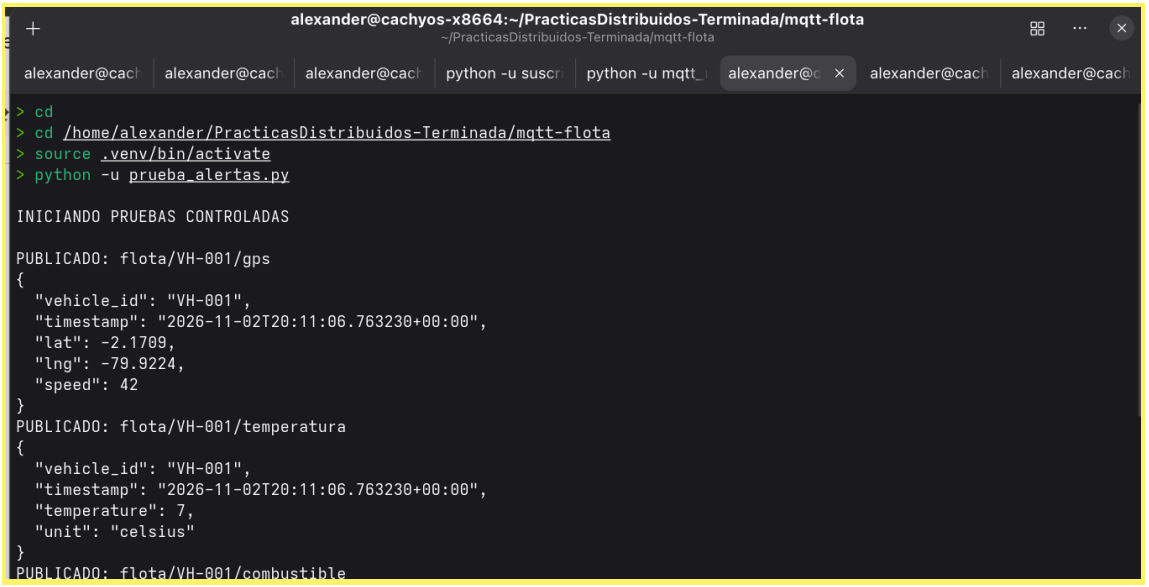


Fig. 26 - Prueba controlada con 7 °C de temperatura y 15 % de combustible para VH-001.

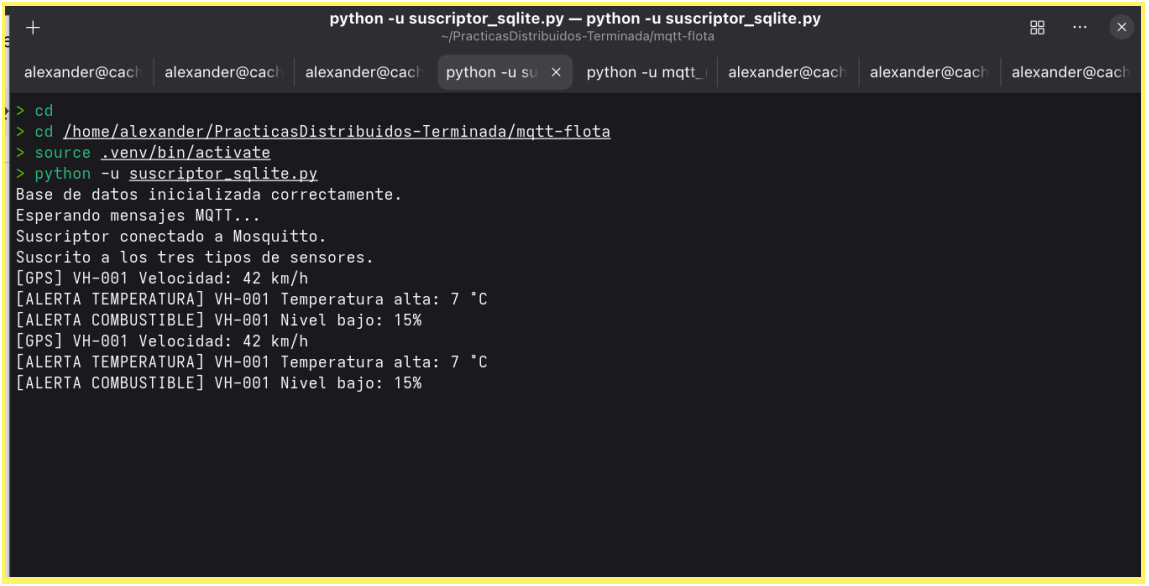


Fig. 27 *- Suscriptor detectando la temperatura alta (7 °C) y el combustible bajo (15 %) de VH-001.*

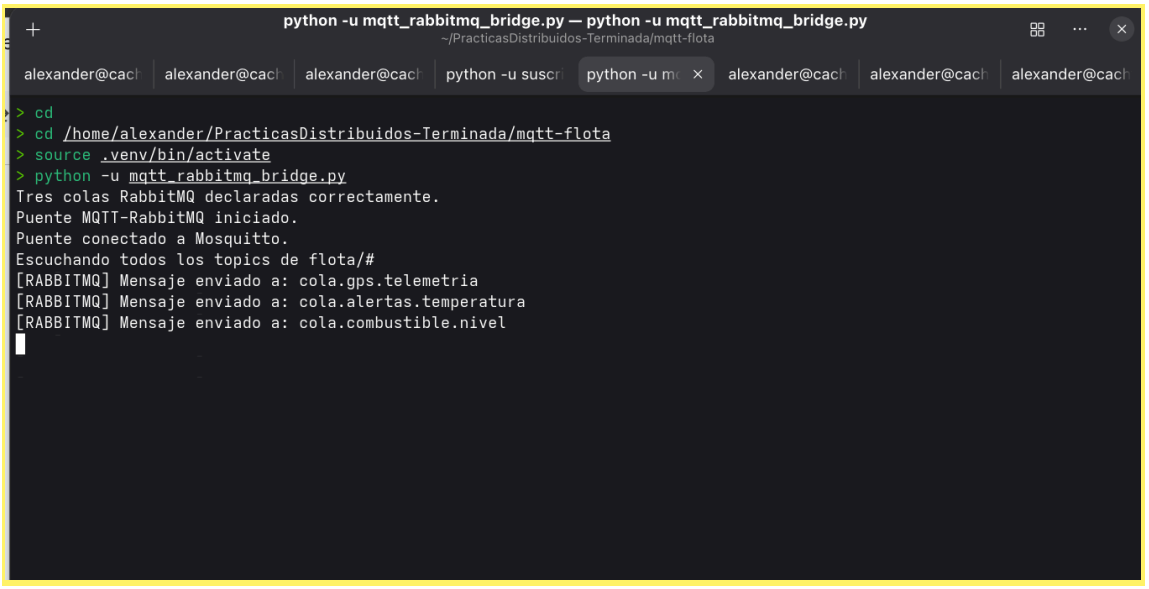


Fig. 28 *- Puente corregido enviando un mensaje a cada una de las tres colas de RabbitMQ.*

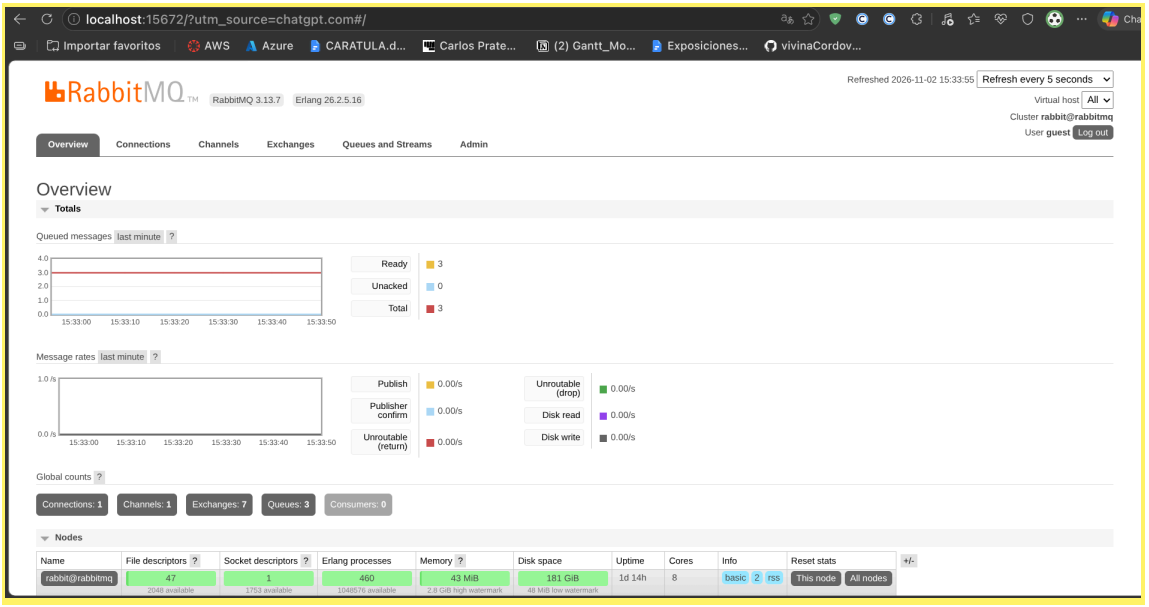


Fig. 29 - *Panel de RabbitMQ con 3 colas, 1 conexión activa y 3 mensajes en estado Ready.*

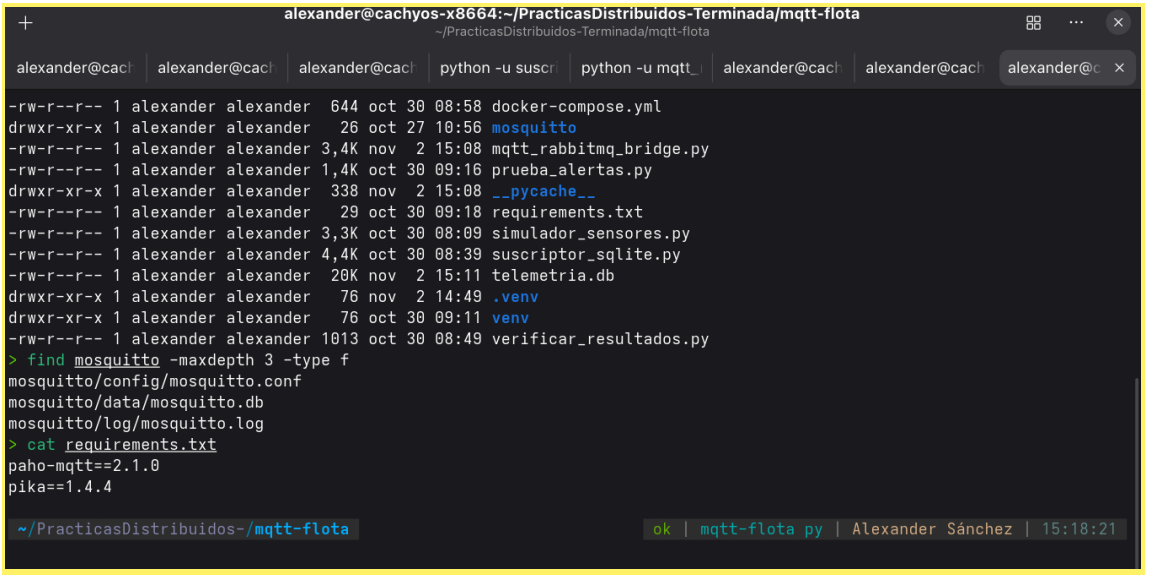


Fig. 30 *- Archivos del broker (configuración, datos y log) y contenido de requirements.txt.*



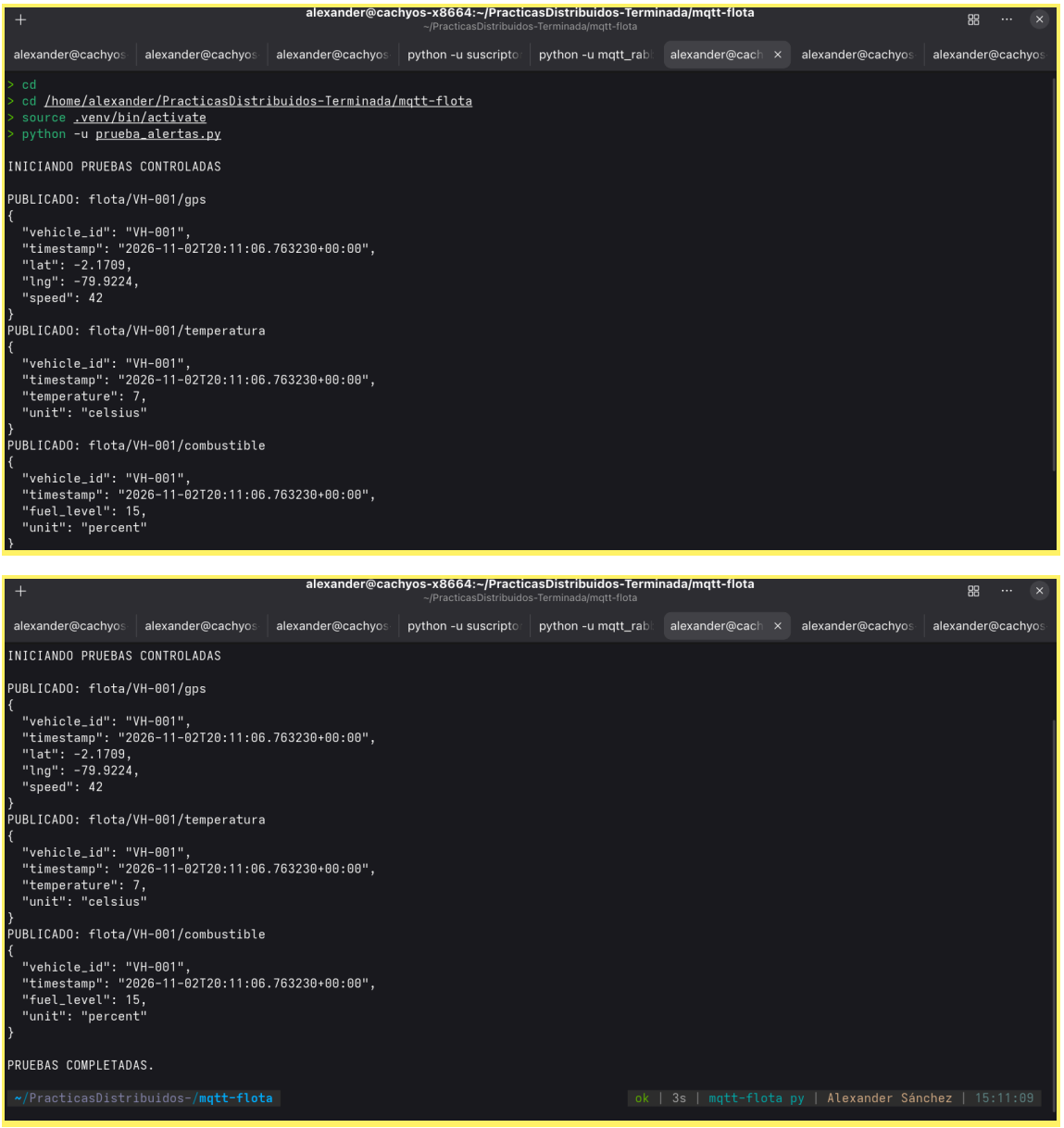


Fig. 31 *- Verificación de la base SQLite con 8 registros por tabla, incluidas las dos lecturas de la prueba controlada.*

La prueba confirmó el flujo completo: el suscriptor registró los datos y mostró las dos alertas, y el puente envió un mensaje a cada cola. En el panel quedaron 3 mensajes en estado Ready y ningún consumidor, lo cual es lo esperado porque los microservicios que leerán esas colas se construyen en la Parte 2.

6. Preguntas de Control

1. Explique las diferencias clave entre MQTT y AMQP en términos de protocolo, patrones de comunicación y casos de uso típicos. ¿Por qué MQTT es más adecuado para IoT?

MQTT es un protocolo muy liviano: el encabezado mínimo de un mensaje ocupa solo 2 bytes y su funcionamiento se reduce a que un cliente publica en un topic y el broker lo entrega a quienes estén suscritos. AMQP, que es el protocolo que usa RabbitMQ, es más completo. Los mensajes llegan a un exchange que, según

unas reglas llamadas bindings, los coloca en una o varias colas, y desde ahí los consumidores los retiran y confirman. Esto permite repartir trabajo entre varios consumidores, guardar mensajes en disco o enrutar con más detalle, pero también exige más configuración y más recursos.

En cuanto a usos, MQTT se ve en sensores, domótica, rastreo GPS y en general en equipos pequeños con conexiones inestables. AMQP aparece más dentro de los sistemas de una empresa, por ejemplo para comunicar microservicios o procesar pedidos y pagos, donde no se puede perder ningún mensaje.

MQTT encaja mejor en IoT porque un sensor o un microcontrolador tiene poca memoria, poca batería y a veces mala señal. Con MQTT el dispositivo solo mantiene una conexión y envía mensajes cortos, y el protocolo ya trae opciones pensadas para eso, como el mensaje de última voluntad (last will) y los niveles de QoS. En la práctica se notó: el simulador solo necesitó saber en qué topic publicar, sin declarar colas ni nada adicional.

2. ¿Por qué MQTT utiliza jerarquía de topics mientras que RabbitMQ utiliza exchanges, colas y bindings? ¿Cuál es la ventaja de cada enfoque?

En MQTT el topic es la dirección del mensaje. Como es un texto separado por "/", la información se puede ordenar como una ruta de carpetas, por ejemplo flota/VH-001/temperatura, y quien se suscribe elige lo que quiere recibir usando comodines: flota/+/temperatura trae la temperatura de todos los vehículos y flota/VH-001/# trae todo lo de un solo vehículo. La ventaja es que no hay que crear nada antes; el topic existe en cuanto alguien publica en él, lo cual es cómodo cuando hay muchos dispositivos.

RabbitMQ separa más las piezas. El productor envía al exchange, el exchange decide con los bindings y la routing key a qué cola va el mensaje, y la cola lo guarda hasta que un consumidor lo procesa. Hay que declarar más cosas, pero a cambio el mensaje no se pierde si el consumidor está apagado, varios consumidores pueden repartirse una misma cola y se puede cambiar el enrutamiento sin tocar a quien publica. Por eso en el sistema de la flota se usan los dos: MQTT para recoger los datos desde los vehículos y RabbitMQ para que los servicios internos procesen las alertas con más garantías.

3. Describa los tres niveles de QoS en MQTT (0, 1, 2). ¿Qué ventajas y desventajas tiene cada uno? ¿En qué escenario del monitoreo de flota utilizaría cada nivel?

QoS 0 (como máximo una vez): el mensaje se envía y no se espera confirmación. Es el más rápido y el que menos tráfico genera, pero si la conexión falla el dato se pierde. Lo usaría para la posición GPS, que se envía cada pocos segundos; si se pierde una lectura, enseguida llega otra.

QoS 1 (al menos una vez): el emisor guarda el mensaje hasta recibir un PUBACK y, si no le llega, lo reenvía. Asegura la entrega, aunque el mensaje puede llegar duplicado. Es el nivel que usé en el simulador y me parece adecuado para la temperatura de la carga y el nivel de combustible, porque de esos datos salen las alertas y un duplicado se puede descartar revisando el timestamp.

QoS 2 (exactamente una vez): usa un intercambio de cuatro mensajes (PUBLISH, PUBREC, PUBREL y PUBCOMP) para que el mensaje llegue una sola vez. Es el más seguro, pero también el más lento y el que más carga pone en el dispositivo y en la red. En la flota lo dejaría para mensajes que no se pueden perder ni repetir, como la confirmación de entrega de un pedido o una orden remota enviada al vehículo.

7. Conclusiones

- Levantar Mosquitto y RabbitMQ con Docker Compose permitió tener los dos brokers funcionando en pocos minutos y con la misma configuración cada vez, sin instalar nada directamente en el sistema operativo.
- La jerarquía flota/{vehículo}/{sensor} resultó práctica: con un mismo esquema de topics el suscriptor separó los datos por tipo de sensor y el puente pudo escuchar todo con flota/#.
- El suscriptor guardó la telemetría en SQLite en tablas separadas (6 registros por tabla tras dos rondas) y detectó la temperatura de 5.0 °C de VH-001 como fuera del rango de la cadena de frío.
- El servicio puente unió la capa IoT con RabbitMQ: en la prueba final envió un mensaje a cada una de las tres colas durables, que quedan listas para que en la Parte 2 los microservicios en Spring Boot consuman las alertas.
- Cuando un mismo programa usa dos librerías con conexiones propias, como paho-mqtt y pika, hay que atender los eventos de ambas. Dejar el control solo a loop_forever() hizo que RabbitMQ cerrara la conexión por falta de heartbeats, y el problema desapareció al procesar los eventos de pika dentro del ciclo principal.

8. Recomendaciones

- Agregar en mosquitto.conf el listener del puerto 9001 para WebSockets, que la guía menciona, y así poder conectar más adelante el dashboard web.
- Activar usuario y contraseña en Mosquitto y cambiar las credenciales guest de RabbitMQ antes de usar el sistema fuera de un entorno de pruebas.
- Mantener un script con valores controlados, como prueba_alertas.py, porque con datos aleatorios la alerta de combustible bajo casi no aparece.
- En el puente, procesar siempre los eventos de pika con process_data_events() (o usar una conexión por hilo) y agregar reconexión automática, para que RabbitMQ no cierre la conexión por falta de heartbeats y no se pierdan alertas.
- Guardar el proyecto en un repositorio Git en lugar de mantener copias de la carpeta, porque en esta práctica una copia incompleta hizo perder tiempo antes de las pruebas finales.

9. Bibliografía / Referencias

[1] J. O. Guamán Q., "Guía de Actividades Práctico-Experimentales Nro. 005: Mensajería distribuida con RabbitMQ y MQTT," Carrera de Computación, Universidad Nacional de Loja, Loja, Ecuador, 2026.

[2] A. Banks, E. Briggs, K. Borgendale y R. Gupta, Eds., "MQTT Version 5.0," OASIS Standard, mar. 2019. [En línea]. Disponible: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[3] Eclipse Foundation, "Eclipse Mosquitto documentation." [En línea]. Disponible: https://mosquitto.org/documentation/ (accedido: 30 oct. 2026).

[4] Eclipse Foundation, "paho-mqtt 2.1.0," Python Package Index. [En línea]. Disponible: https://pypi.org/project/paho-mqtt/ (accedido: 30 oct. 2026).

[5] RabbitMQ, "AMQP 0-9-1 Model Explained." [En línea]. Disponible: https://www.rabbitmq.com/tutorials/amqp-concepts (accedido: 30 oct. 2026).

[6] Pika, "Introduction to Pika." [En línea]. Disponible: https://pika.readthedocs.io/ (accedido: 30 oct. 2026).

[7] Docker Inc., "Docker Compose." [En línea]. Disponible: https://docs.docker.com/compose/ (accedido: 30 oct. 2026).

[8] SQLite, "SQLite Documentation." [En línea]. Disponible: https://www.sqlite.org/docs.html (accedido: 30 oct. 2026).

10. Anexos

Anexo A. Archivos del proyecto mqtt-flota

- docker-compose.yml: definición de los servicios mosquitto y rabbitmq.
- mosquitto/config/mosquitto.conf: configuración del broker MQTT.
- simulador_sensores.py: publicador de telemetría de los tres vehículos.
- suscriptor_sqlite.py: suscriptor que guarda los datos en SQLite y muestra alertas.
- verificar_resultados.py: consulta rápida de los registros guardados.
- mqtt_rabbitmq_bridge.py: servicio puente entre Mosquitto y RabbitMQ.
- prueba_alertas.py: publicación de valores controlados para forzar alertas.
- requirements.txt: dependencias de Python (paho-mqtt 2.1.0 y pika 1.4.4).
- telemetria.db: base de datos SQLite generada por el suscriptor.

Anexo B. Orden de ejecución

- docker compose up -d
- python suscriptor_sqlite.py
- python -u mqtt_rabbitmq_bridge.py
- python simulador_sensores.py --rondas 2
- python prueba_alertas.py
- python verificar_resultados.py